

Practice Assignment 7

LIBRARY MANAGEMENT SYSTEM

Following previous Practice Assignment

Unit Test & Integration Test

Learning Objectives

By completing this exercise, you will:

- Write **unit tests** for controllers using **xUnit** and **Moq**
- Implement **integration tests** using **TestServer**
- Test controller logic, service layers, and complete API workflows
- Understand testing best practices in .NET applications

Required Setup

1. Install Required NuGet Packages

Install the following packages in your test project:

package xUnit

package xUnit.runner.visualstudio

package Microsoft.NET.Test.Sdk

package Moq

package Microsoft.AspNetCore.Mvc.Testing

2. Create Test Project

dotnet new xunit -n Library.Api.Tests

dotnet add Library.Api.Tests reference Library.Api/Library.Api.csproj

Exercise 1: Unit Testing - BooksController (40 points)

Requirements:

Create unit tests for the BooksController class. Your controller should have these methods:

- GetBookById(int id) - Returns a book by ID

- GetAllBooks() - Returns all books
- CreateBook(Book book) - Creates a new book
- DeleteBook(int id) - Deletes a book

Tasks:

1. **Test GetBookById method** (10 points)
 - o Test case: Returns NotFound when book doesn't exist
 - o Test case: Returns correct book when it exists
2. **Test GetAllBooks method** (10 points)
 - o Test case: Returns empty list when no books exist
 - o Test case: Returns all books when books exist
3. **Test CreateBook method** (10 points)
 - o Test case: Returns CreatedAtAction with valid book data
 - o Test case: Returns BadRequest with invalid book data
4. **Test DeleteBook method** (10 points)
 - o Test case: Returns NoContent when book exists and is deleted
 - o Test case: Returns NotFound when book doesn't exist

Requirements:

- Use **Moq** to mock LibraryDbContext
- Follow **Arrange-Act-Assert** pattern
- Use meaningful test method names
- Create file: BooksControllerTests.cs

Exercise 2: Integration Testing - API Endpoints (40 points)

Requirements:

Create integration tests that test the complete HTTP request/response pipeline.

Tasks:

1. **Create CustomWebApplicationFactory** (10 points)
 - o Configure in-memory database
 - o Seed test data
 - o Override services for testing
2. **Test GET endpoints** (10 points)
 - o GET /api/books returns 200 OK
 - o GET /api/books/{id} returns 200 OK when book exists

- o GET /api/books/{id} returns 404 when book doesn't exist
- 3. **Test POST endpoint** (10 points)
 - o POST /api/books returns 201 Created with valid data
 - o POST /api/books returns 400 Bad Request with invalid data
- 4. **Test DELETE endpoint** (10 points)
 - o DELETE /api/books/{id} returns 204 No Content when successful
 - o DELETE /api/books/{id} returns 404 when book doesn't exist

Requirements:

- Use WebApplicationFactory<Program>
- Test actual HTTP requests and responses
- Verify response status codes and data
- Create file: BooksApiIntegrationTests.cs

Exercise 3: Authentication Testing (Bonus - 20 points)

Requirements:

Create integration tests for JWT authentication flow.

Tasks:

1. **Test Login endpoint** (5 points)
 - o POST /api/auth/login returns token with valid credentials
 - o POST /api/auth/login returns 401 with invalid credentials
2. **Test Protected endpoints** (10 points)
 - o Protected endpoint returns 401 without token
 - o Protected endpoint returns 200 with valid token
3. **Test Role-based access** (5 points)
 - o Admin endpoint returns 403 for regular user
 - o Admin endpoint returns 200 for admin user

Requirements:

- Test JWT token generation and validation
- Test authorization headers
- Create file: AuthenticationIntegrationTests.cs

Expected Project Structure

Library.Api.Tests/

├── Controllers/

```
|   └─ BooksControllerTests.cs
|   └─ Integration/
|       └─ BooksApiIntegrationTests.cs
|       └─ AuthenticationIntegrationTests.cs (bonus)
|       └─ CustomWebApplicationFactory.cs
|       └─ Library.Api.Tests.csproj
```

Submission Requirements

Required Deliverables (100 points total):

1. **Test Project Files** (80 points)
 - o BooksControllerTests.cs with all required unit tests
 - o BooksApiIntegrationTests.cs with all required integration tests
 - o CustomWebApplicationFactory.cs for test setup
2. **Documentation** (20 points)
 - o Screenshots showing all tests passing (dotnet test)
 - o Brief explanation of your testing approach (200-300 words)
 - o Comments in code explaining complex test scenarios
3. **Bonus** (+20 points)
 - o Complete JWT authentication testing
 - o Additional edge case tests
 - o Test coverage report

Grading Criteria:

- **Functionality (50%)**: All tests pass and cover required scenarios
- **Code Quality (30%)**: Clean, readable code with proper naming
- **Testing Best Practices (20%)**: Proper use of AAA pattern, mocking, assertions

Testing Commands

Run all tests

```
dotnet test
```

Run with detailed output

```
dotnet test --verbosity normal
```

Run specific test class

```
dotnet test --filter "BooksControllerTests"
```

Tips:

1. **Start Simple:** Begin with basic test cases, then add complexity
2. **Mock Dependencies:** Use Moq to isolate units under test
3. **Test Edge Cases:** Don't forget null values, empty collections, invalid IDs
4. **Meaningful Names:** Test names should describe what they're testing
5. **One Assert Per Test:** Keep tests focused on single behavior
6. **Arrange-Act-Assert:** Structure all tests consistently